Embedded Systems Design

2026 SPRING

JINHO LEE

ECE, SNU

Accelerated
Intelligent
Systems Lab.

WARNING – Challenge Awaiting

- This course is known for being extremely tough
- We will do a lot of work, with a homework on-going all the time

- Prerequisites
  - Computer architecture
  - Fluency in C++

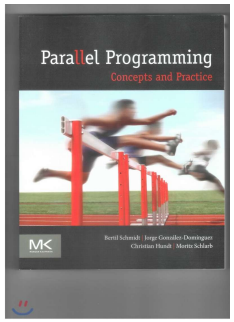
- If you have any information regarding this class from 2024..
  - No mandatory labs
  - But mostly the same contents as homeworks
  - Which are easier by
    - Better (and easier) baseline materials
    - Lower perf bars
    - Shorter and less important written reports
  - <u>But it will still be tough</u>

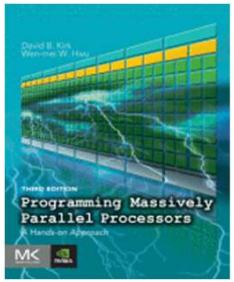
Class info

- Embedded Systems Design
- M/W 5:00pm – 6:15pm @ 301-302 (Lecture)
- Office hour
  - send mails for appointments
- Announcements, lecture notes:
  - eTL (Make sure to update)
- Instructor
  - Jinho Lee
    - AI system optimization
    - Computer architecture
    - Model compression
    - Always hiring Grad students / Undergrad interns
  - 301-1009
  - leejinho@snu.ac.kr (Lectures, or anything else)
  - **Please DO NOT send anything to TA's personal mails!**
- TAs
  - emb.2026.spring@gmail.com (shared, preferred)

Textbooks

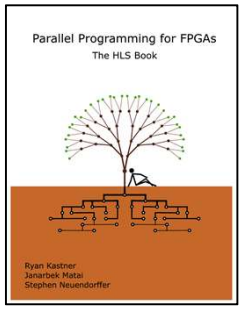
- <Parallel Programming: Concepts and Practice : Concepts and Practice>, 2nd edition
  - For multithread programming and parallel algorithms





- <Programming massively parallel processors>, 3rd edition
  - For CUDA programming

- <The HLS book>
  - HLS for FPGAs
  - http://kastner.ucsd.edu/hlsbook/



- Materials from previous lectures
  - Videos: https://drive.google.com/open?id=19DmGbtMFKkC1-JUYeCAepuTUuw3ijZXh&usp=drive_fs
  - Lecture notes: https://drive.google.com/open?id=18tY1XPPklTzx-J1JwR4iPqgxWuE0QJoz&usp=drive_fs

Typo alert!

- Many of the slides are being created and used for the second time (some are being recycled)
- There could be (and will be) still typos/errrors
- Small extra credits will be given to the <u>first</u> student who reports them
  - Varying points (1-3) based on how severe the error is
  - From simple typos to logical errors, inconsistency, and mathematical errors
  - Use the error report board (eTL) or simply raise your hand during lectures

Course Schedule

1. Embedded system with CPUs + SIMD (Week 1~4)

2. Embedded GPUs with CUDA (Week 5-8)

3. Custom IP with HLS (Week 9-12)

4. Others, including Doom (Week 13-15)

Lecture/Lab (Tentative)

Week	Lecture	Category	Homework	Other
1	Intro to embedded, basic multithreading	Embedded Processors	Parallel vadd/matmul	
2	Optimizing for memory bandwidth		Parallel matmul w/ SIMD	
3	Embedded inference/training for ML		Diffusion on CPU	
4	Case study: Prefix sum			
5	Embedded GPUs, CUDA basics	Embedded GPUs	CUDA – vadd/matmul	
6	GPU application study: reduction		CUDA – Reduction	
7	GPU optimization techniques		Diffusion on GPU	
8	Additional techniques: multi-stream, kernel fusion, etc			Absent (DATE 2026)

Lecture/Lab (Tentative)

Week	Lecture	Category	Homework	Other
9	Embedded IP design	Custom IP and HLS	HLS vadd/matmul	Midterm (4/29 6pm)
10	HLS basics		Dataflow design	
11	HLS optimization techniques		Diffusion on FPGA / HLS (likely a team assignment)	
12	Application study: FIR filter			Absent (MLSys 2026)
13	Tiny embedded devices	Potpourri	Porting Doom	
14	ML for embedded (TinyML, federated learning)		-	
15	Other embedded platforms		-	Final (6/17 6pm)

Midterm / Final Exams

- Midterm
  - 4/29 6pm – 9pm
- Final
  - 6/17 6pm – 9pm
- Live coding + written exam
  - You will need a laptop – ask for help if you don't have one

Course Schedule

- Major homeworks -

1. Diffusion model inference <u>on CPU</u>
   - Use multithreading, SIMD, etc

2. Diffusion model inference <u>on CPU + GPU</u>
   - Accelerate certain kernels in GPU

3. Diffusion model inference <u>on CPU + FPGA</u>
   - Design custom IPs on FPGA using HLS

- All projects are done over the same codestack
- Same goal, same code, just different hardware
- Success of earlier projects will be carried to the later ones

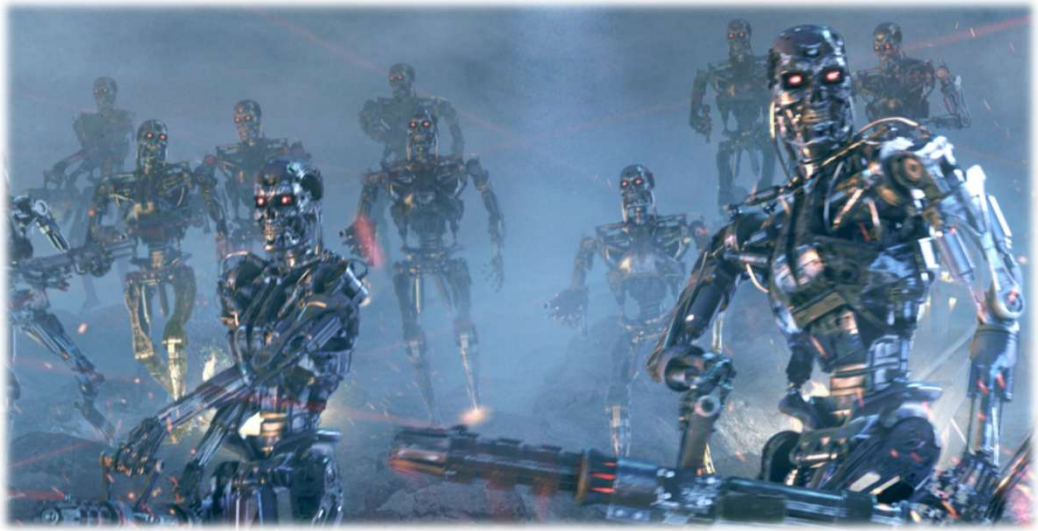
When you have a question..

- The best way: raise your hand
  - Everyone gets to hear the answer
  - Using Korean for questions is totally fine

- Second best way: Post it to the eTL **QnA board**
  unless you have a reason (e.g. Privacy, etc)
  - To avoid me/TAs having to answer similar questions multiple times

- Not the best, but a possible way: Using personal mail
  - When using mails or private post of the QnA board, If you want me not to disclose the question, please explicitly say so.
  - Otherwise, **I will assume that I can open them in public** if I think it's okay

Rise of the Machines



- Using AI language models (e.g., gpt) to generate code is allowed in this course
  - Because it's an advanced-level class
  - But not recommended – you should be able to do that by yourself, and at least verify your intent has been correctly reflected in the code
- Policy
  - **Attach Usage Logs**: You must submit all AI interaction logs used for your work.
  - **Reproducibility**: The logs should be detailed enough to make your process reproducible.
  - **Plagiarism Warning**: If your submission overlaps with a colleague's and you have not provided the supporting logs, it will be treated as full plagiarism.
- Same goes to the report. You can freely use AI models for your report, but cleanly state what part you used them for, and provide the usage log.

Plagiarism Alert!

- A very serious cheating
- The department is placing a special attention to plagiarism issues
- If found, everyone involved will be referred to the disciplinary committee
  - This is real, not just an empty threat

- The moment you look at other's code in any way, that becomes cheating
- (!) You have an obligation to maintain security of your code
  - Do not upload your code in a public repo
  - **When leaving your computer, lock your screen,** so that no one can access it
  - You are guilty if you leave anything unattended

This is an English medium class!

- This class is taught in English
  (if you haven't noticed yet)

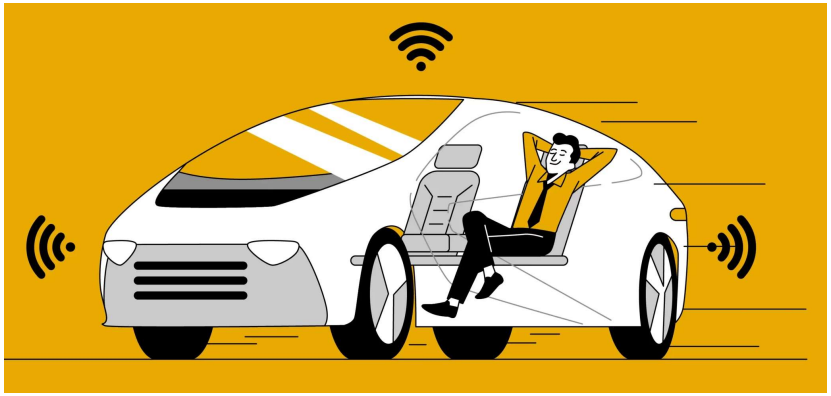
- You can ask questions in Korean
  - I will translate the question to English,
    and answer in English

- Answers for HW/Quiz/Exams allow Korean
  - But I will give the materials only in English
  - No separate Korean material provided

Ready?

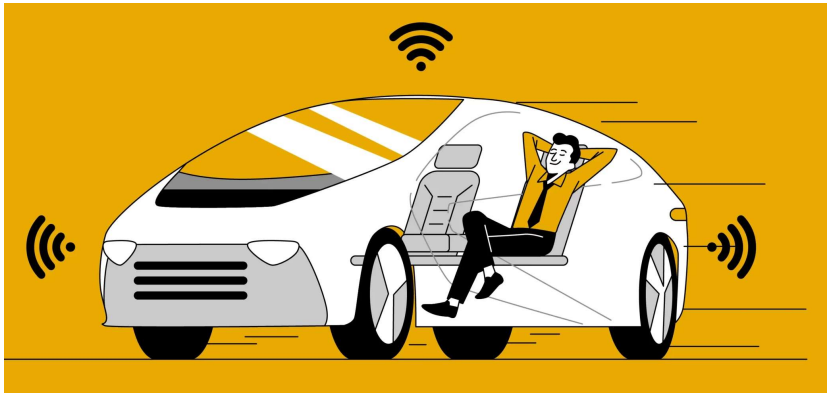
What are Embedded Systems?

- Definition of embedded systems is often unclear, which only makes sense for those who already know it
- Examples:



What are Embedded Systems?

- Here's my try: An embedded system is <u>a computing system</u> with
  1. Limited price, power and size
  2. With a specialized function set



General-purpose Computer

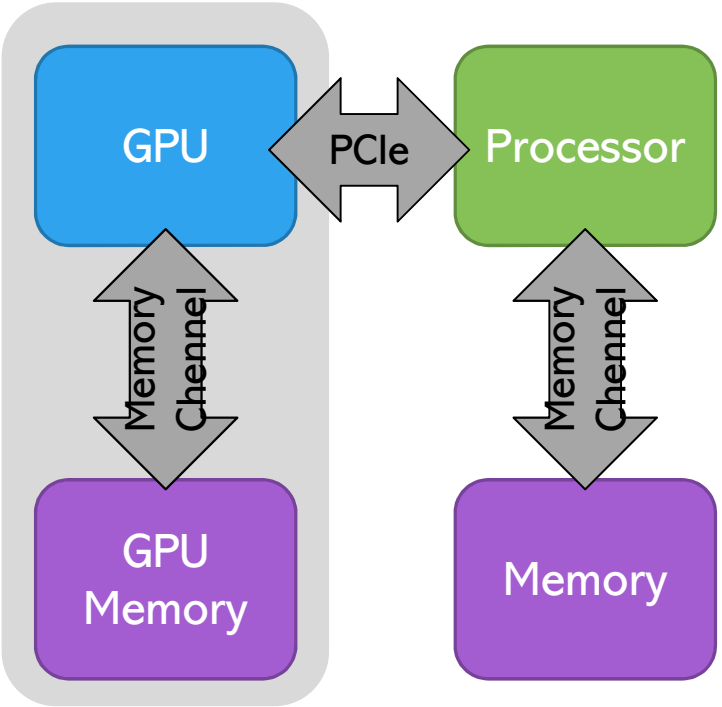
- This is a general purpose computer

General-purpose Computer

- When a general-purpose computer is used for devices that need computing power…

General-purpose Computer (Desktop, Server)

- For relatively high-speed, high price



Processor
- Usually X86 (Intel, AMD)
- Out-of-order
- CISC
- 4-100 cores

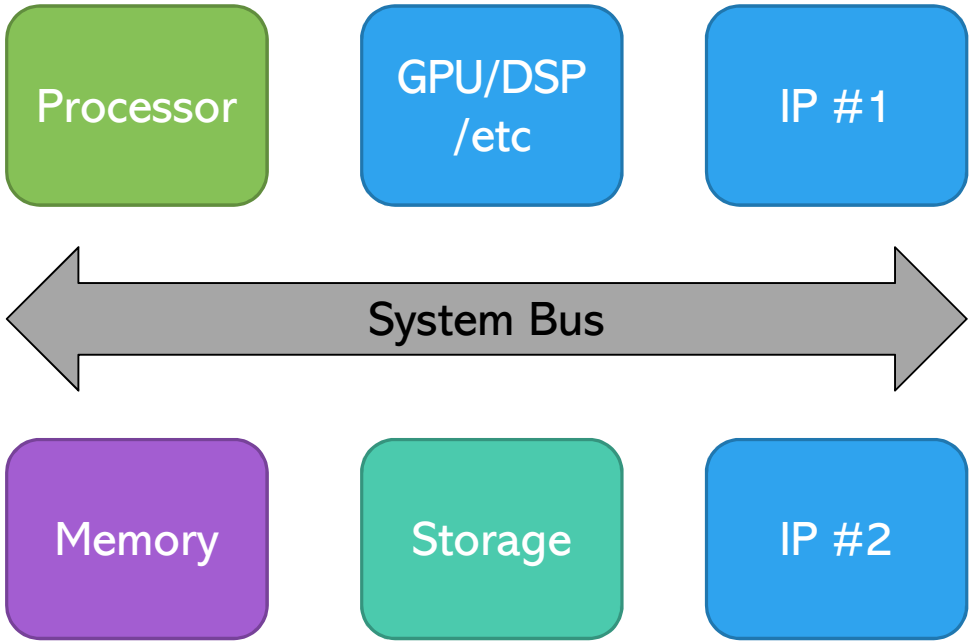
Memory
- Usually DDR4/DDR5
- Attached to mainboard
- Capacity 4GB ~ 1TB

GPU
- Usually a separate card thru PCIe
- Capacity 10~100GB
- Own memory (GDDR, HBM) with high-bandwidth

Embedded Systems (Simplified)

Processor
- Usually ARM
- In-order/Out-of-order
- RISC
- 4-16 cores

GPU/DSP
- Accelerator
- Small-sized
- Often on the same chip with the processor



Custom IPs
- Depending on target functionality
- Communication, control, sensors, audio, etc

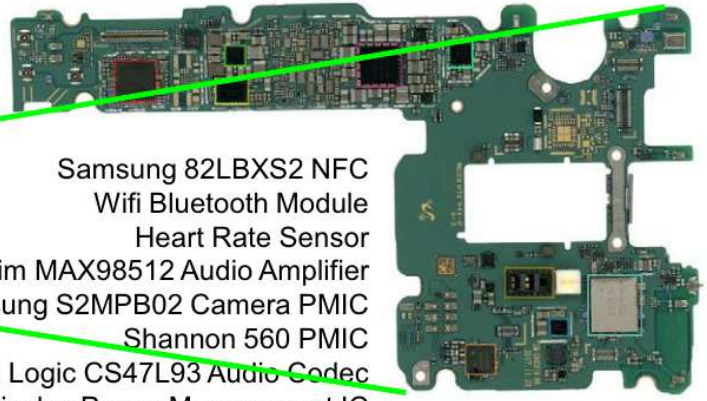
Memory
- Usually LPDDR (on-board)
- Capacity 1-32GB
- Shared for all components

- **Everything typically on-board or on-chip**

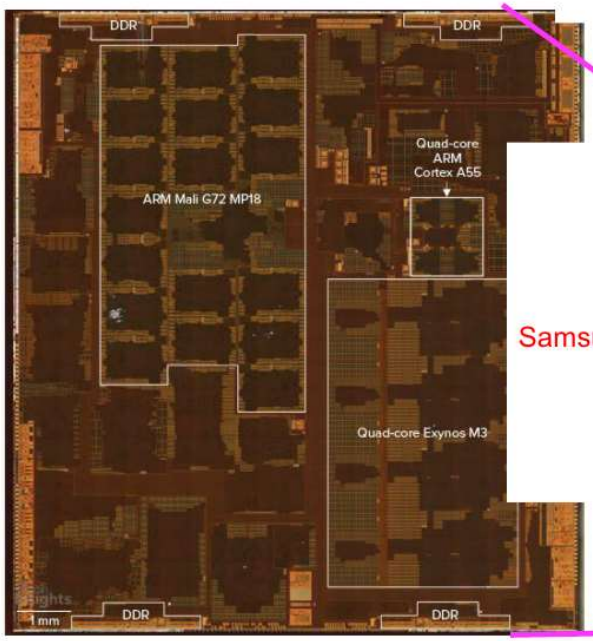
Embedded System Example

- **Smart phone**

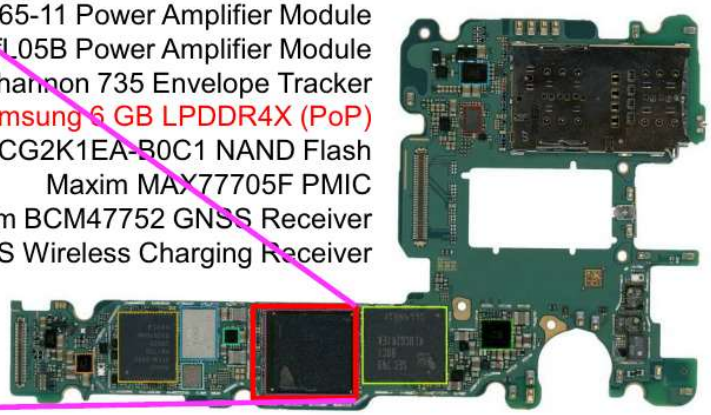
Galaxy S9



Samsung 82LBXS2 NFC
Wifi Bluetooth Module
Heart Rate Sensor
Maxim MAX98512 Audio Amplifier
Samsung S2MPB02 Camera PMIC
Shannon 560 PMIC
Cirrus Logic CS47L93 Audio Codec
Samsung S2DOS05 Display Power Management IC
Shannon 965 RF Transceiver

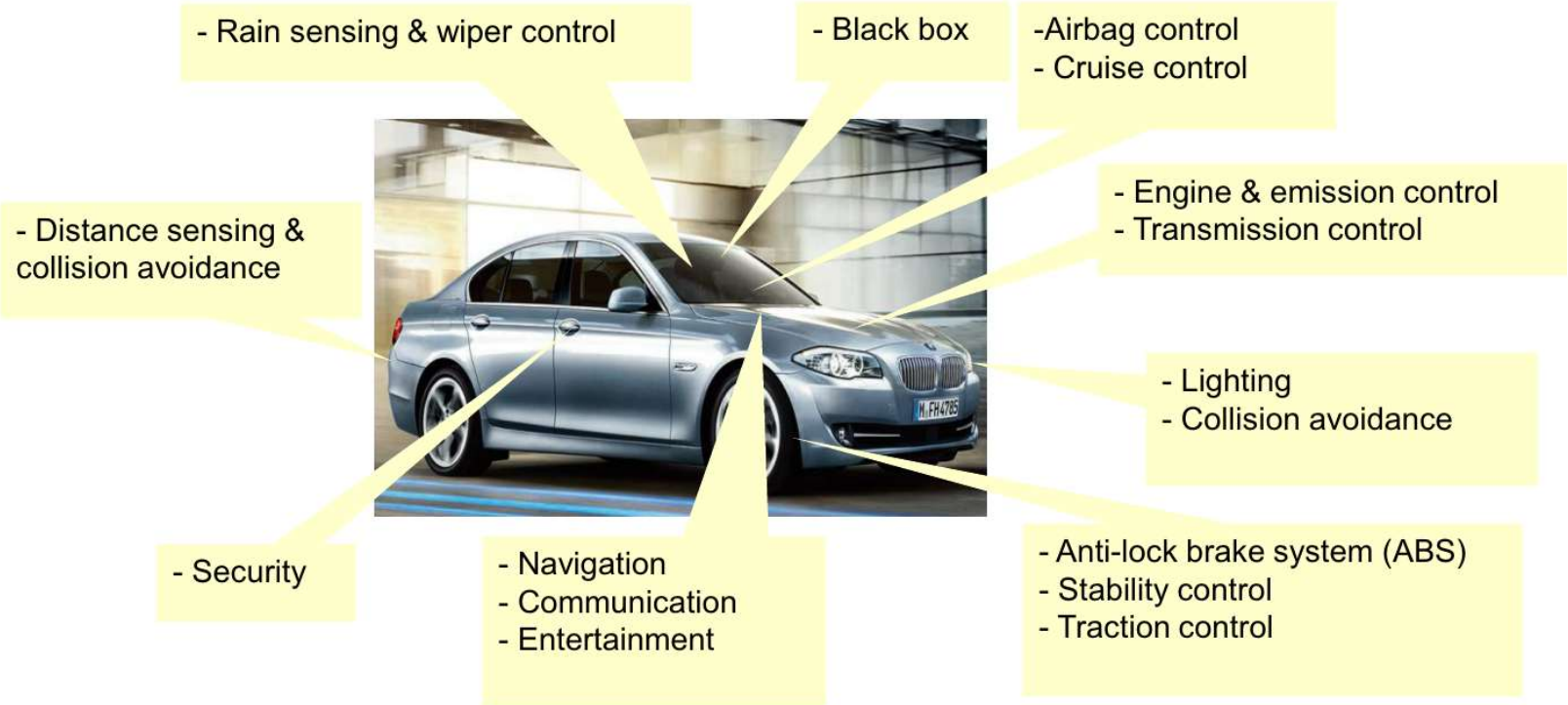


Maxim MAX98512 Audio Amplifier
Avago AFEM-9090 Front End Module
Skyworks SKY77365-11 Power Amplifier Module
Murata fL05B Power Amplifier Module
Shannon 735 Envelope Tracker
Samsung Exynos 9810 + Samsung 6 GB LPDDR4X (PoP)
Samsung KLUCG2K1EA-B0C1 NAND Flash
Maxim MAX77705F PMIC
Broadcom BCM47752 GNSS Receiver
IDT P9320S Wireless Charging Receiver



Embedded System Example

- Vehicles
- Today's (non-self-driving) automobiles already have 100+ separate embedded systems
- Each runs with a processor



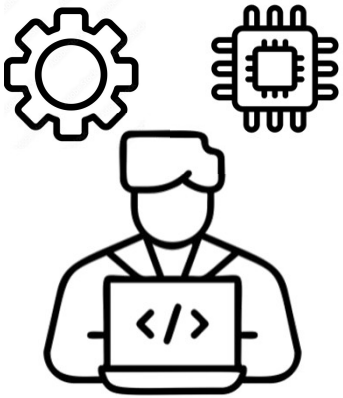
Designing an Application

- General-purpose computer

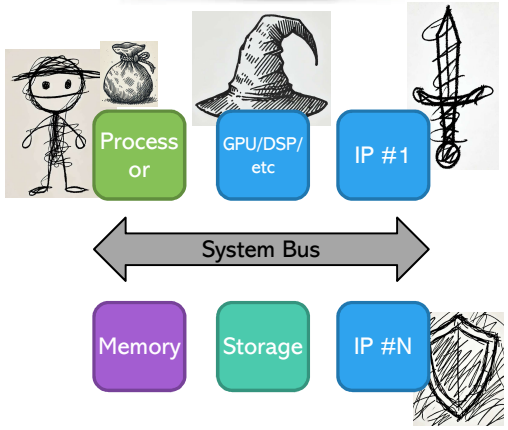
SW programming

SW Application

Functionality



SW/HW codesign



Embedded System

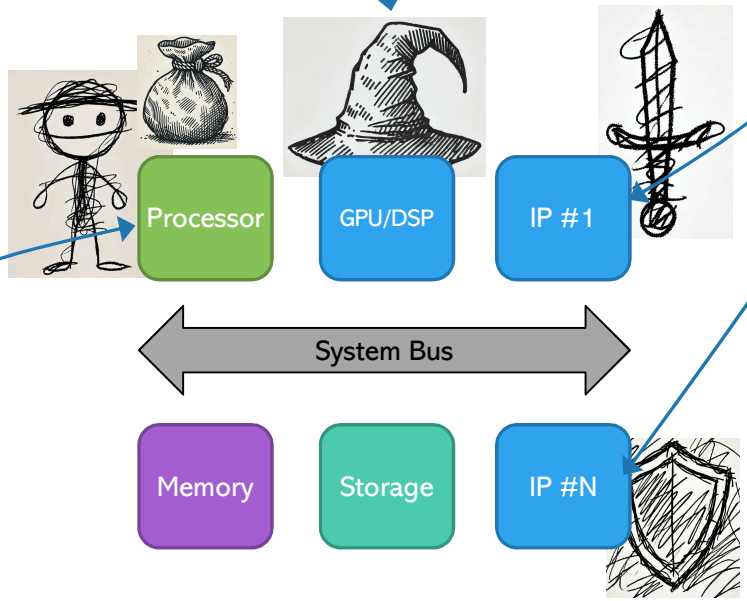
- Embedded system

SW/HW Codesign

- Embedded system is often designed with the SW/HW codesign approach

A part that I want to execute in a GPU

Parts that I want to design custom hardware IPs



Parts that I want to execute in a CPU

SW/HW Codesign

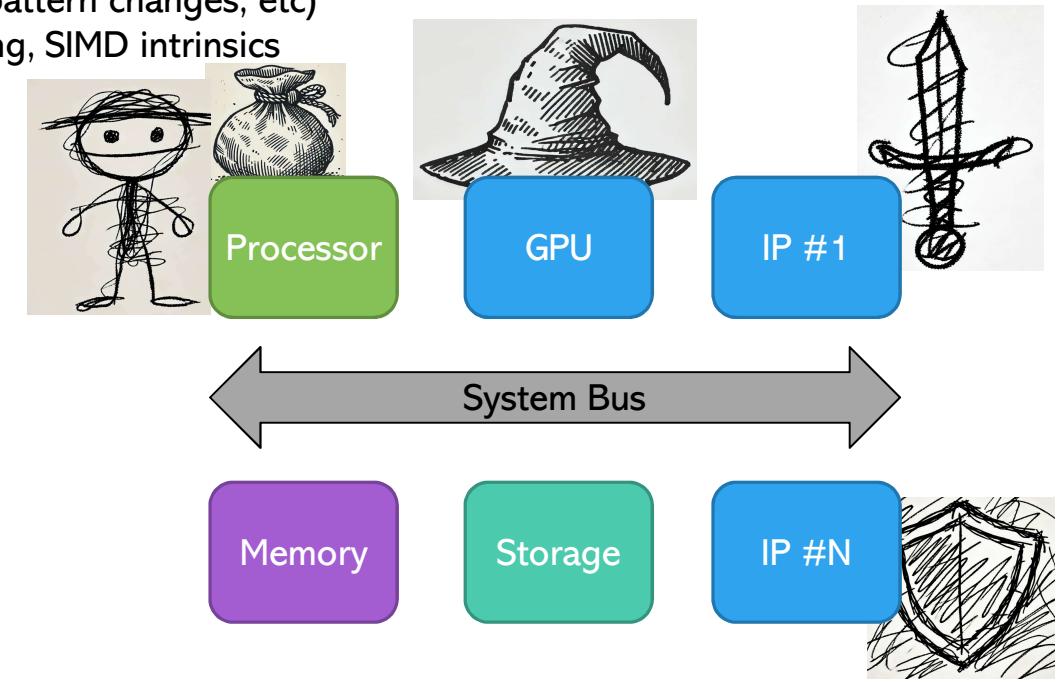
- Embedded system is often design with the SW/HW codesign approach

Processor

- When speed is less important, and time-to-market is more important
- Flexibility is needed (updates, user pattern changes, etc)
- → Multithreading, SIMD intrinsics

GPU

- When speed is important
- Application fits GPU architecture
- → CUDA



Custom IP

- When speed is very important
- Application needs special hardware
- Flexibility is less important
- → HLS

Course Outline

- Embedded system is often design with the SW/HW codesign approach

Processor

- → Multithreading, SIMD intrinsics
- Week 1-4

GPU

- → CUDA
- Week 5-8

Custom IP

- → HLS
- Week 9-12

Course Outline

- Embedded system is often design with the SW/HW codesign approach

Processor

- → Multithreading, SIMD intrinsics
- Week 1-4

GPU

- → CUDA
- Week 5-8

Custom IP

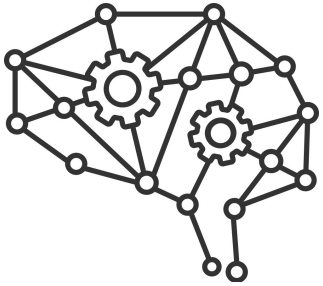
- → HLS
- Week 9-12

Course Outline

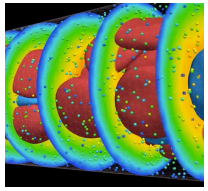
- Embedded system is often design with the SW/HW codesign approach
- Turns out that the individual skillsets are used for any computer systems, not just embedded systems
  - Embedded systems turns out to be a system that you can use them all

Processor
- → Multithreading, SIMD intrinsics
- Week 1-4

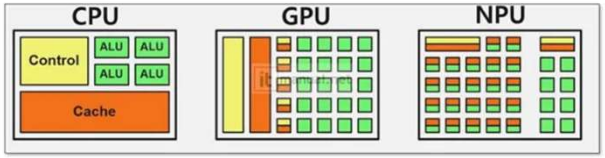
Database


Deep Learning

GPU
- → CUDA
- Week 5-8


Simulation

Custom IP
- → HLS
- Week 9-12



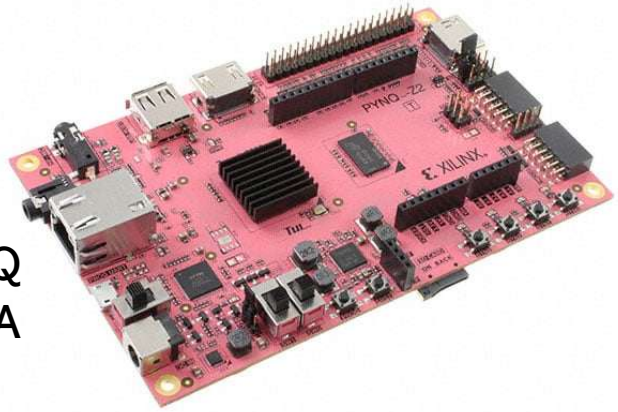
New hardwares

Devices

Jeton orin nano,
6 arm cores + NVIDIA GPU

Processor

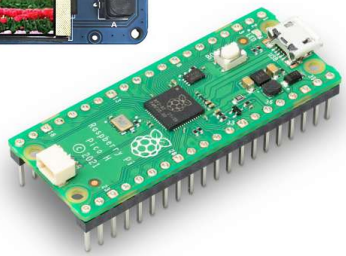
- → Multithreading, SIMD intrinsics
- Week 1-4



PYNQ
2 arm cores + Xilinx FPGA

GPU

- → CUDA
- Week 5-8



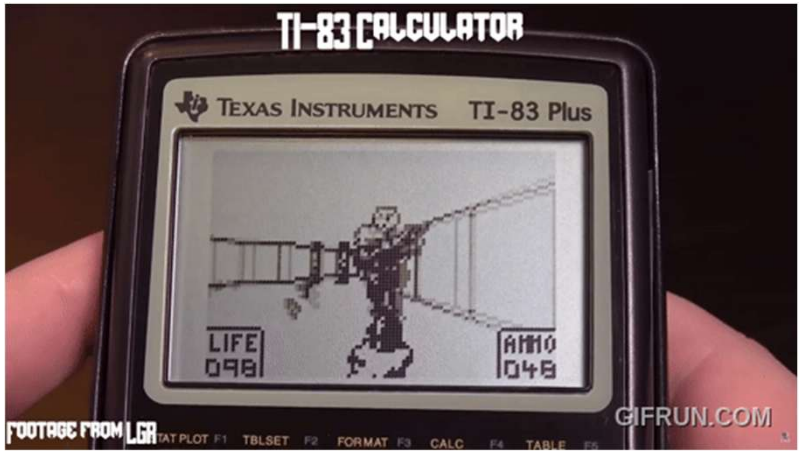
Custom IP

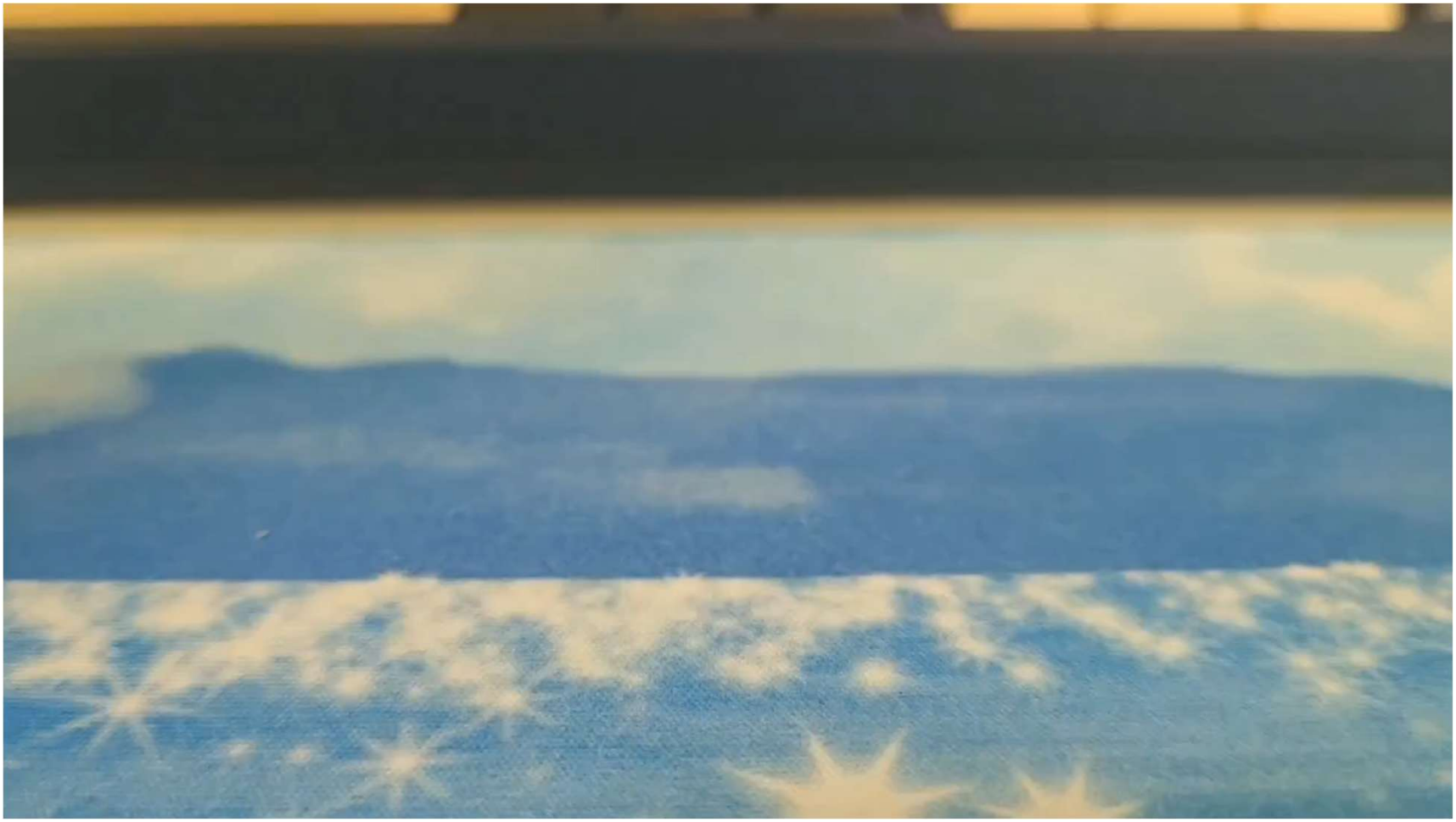
- → HLS
- Week 9-12

Raspberry Pi Pico
A microcontroller @ 133MHz
264KB SRAM + 2MB Flash

Porting Doom

- Have you seen these?
- Doom is easy to port to various devices
  - The code is fully open
  - Has almost no dependency
  - Written in C
  - It was released in 1993, so the required computing power is very low



What we will do

- Raspberry Pi pico + a small gamepad monitor+buttons